Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Full Stack Development using MERN Stack

Name	Priti Yadav
Roll no	T054
Class	TYIT
Academic year	2026-2027
Submitted to	Mrs. Rohini Bhosale

Abstract

CleanCity is a web-based waste complaint and tracking application intended to improve communication between citizens, field employees and a city administration. Citizens can create an account, sign in, submit a waste-related complaint with a photograph and location, and follow the complaint status. Administrators can review complaints, view dashboard counts, add employees, assign complaints to employees and update work status. Employees can access the tasks assigned to them and update progress. The application uses a React frontend and a Node.js/Express backend connected to MongoDB through Mongoose. Authentication is implemented with JSON Web Tokens (JWT), and user passwords are hashed with bcryptjs. The project is deployed as a frontend/backend application, with the backend hosted on Render.

1. Project Title

CleanCity – Waste Management Tracking System for Local Bodies

Project category: Full Stack Development using MERN Stack. The project is developed as a practical web application for reporting and monitoring local waste-cleanup complaints.

2. Objective

The main objective is to develop a web application using React, Node.js, Express.js and MongoDB that supports a digital workflow for reporting waste issues and tracking cleanup activity.

Specific objectives:

- Provide a simple registration and login facility for citizens and employees, with a separate administrator login.
- Allow citizens to submit waste complaints with a category, description, photograph and geographical coordinates/landmark.
- Allow citizens to view the complaints they have submitted and their current status.
- Allow administrators to view complaints, dashboard statistics, add employees and assign complaints.
- Allow authorized employees/administrators to update complaint progress and resolution status.
- Store application records in MongoDB and communicate through REST API endpoints.

3. Project Description

CleanCity provides a common digital interface for residents to report waste accumulation and for local-body staff to coordinate cleanup. The citizen interface contains account registration, login, a report form and a My Reports page. The report form captures the waste category, a description, an image, latitude and longitude, and an optional nearby landmark. The backend stores the uploaded image in the server's uploads directory and stores the complaint record in MongoDB.

The administrator interface presents summary cards for waiting/pending, assigned, in-progress, cleaned/resolved and total complaints. It provides status filters and an employee creation form. An administrator can select an employee and assign a complaint. The backend updates the complaint status and creates notifications for the citizen and assigned employee. Employees can view their assigned tasks and update the status as work progresses.

The project is a complaint and cleanup tracking system. It does not, in the reviewed implementation, establish automatic weighing, smart-bin sensor telemetry, route optimisation or vehicle GPS tracking; these may be considered as future enhancements.

4. Functional Requirements

Citizen functions: register an account; log in; submit a waste complaint; attach a photo; share location; view own complaints; receive notifications about status changes.

Administrator functions: use the administrator login; view all complaints; view complaint statistics; filter complaints by status; create employee accounts; assign complaints to employees; update complaint status; view notifications.

Employee functions: log in as an employee; view assigned tasks; update the status of assigned work. The backend checks that an employee is updating a task assigned to that employee.

System functions: validate required data; store users, complaints and notifications in MongoDB; hash passwords; issue and verify JWTs; store uploaded image files; return suitable HTTP status codes and error messages.

5. Tools and Technologies Used

Frontend: React 18, React DOM, React Router DOM and Vite. The frontend repository package manifest identifies these packages.

Backend: Node.js, Express.js, Mongoose, MongoDB, bcryptjs, jsonwebtoken, dotenv, cors and Multer.

Database: MongoDB, accessed through Mongoose models. MongoDB Compass is used to inspect database collections.

Development and deployment: Visual Studio Code, Git, GitHub and Render. The frontend and backend are maintained in separate GitHub repositories.

6. Technical Requirements and System Architecture

Client layer: the React single-page application renders registration, login, report submission, citizen reports and administration screens. React Router DOM manages navigation.

API layer: the Express application exposes REST endpoints under /api/auth, /api/complaints, /api/employees and /api/notifications. JSON request bodies are parsed with express.json(). Uploaded files are served from /uploads.

Authentication layer: the backend issues a JWT after successful registration/login. Protected API requests use the token through authentication middleware. Role checks distinguish citizen, employee and admin permissions.

Data layer: Mongoose models define User, Complaint and Notification documents. MongoDB stores account information, complaint details, assignment/status information and notification records.

High-level flow: React frontend → HTTP request → Express route → authentication/role middleware → Mongoose model → MongoDB → JSON response → React UI update.

7. Main Modules

7.1 User Authentication and Registration: public sign-up creates a citizen account. Login and admin-login are separate routes. The API returns a token and a public user object containing ID, name, email and role; the password is not included in that public object.

7.2 Complaint Reporting: a signed-in citizen submits category, description, photo, latitude, longitude and optional landmark. The backend accepts an image upload with a maximum size of 5 MB and rejects non-image files. The complaint is stored with the citizen reference and initial pending status.

7.3 Citizen Complaint Tracking: the My Reports endpoint returns complaints belonging to the logged-in citizen, newest first. The frontend displays the complaint photo, category, description, submission time and status.

7.4 Administrator Dashboard: the administrator can view all complaints, request aggregate status counts, filter the list by status and add employee accounts. Dashboard status values are represented internally as pending, assigned, in_progress, resolved and rejected; the interface may display friendlier labels such as Waiting and Cleaned.

7.5 Employee Assignment: the administrator selects a valid employee and assigns the complaint. The backend updates assignedTo and status, and creates notification records for both the citizen and employee.

7.6 Work Status and Notifications: an administrator or the employee assigned to a complaint can update its status. Notifications are stored per user; the notification API returns the latest 30 items and unread count and supports marking notifications as read.

8. Database Design

User collection: name (required), email (required, unique, lowercase), phone, password (required and excluded from normal query selection), role (citizen, employee or admin), createdAt and updatedAt. Passwords are hashed using bcryptjs before saving.

Complaint collection: citizen (reference to User), category (garbage_pile, overflowing_bin, illegal_dumping, missed_pickup or other), description (required, maximum 1,000 characters), photo path, location (lat, lng, landmark), status (pending, assigned, in_progress, resolved or rejected), assignedTo (User reference), note, resolvedAt, createdAt and updatedAt.

Notification collection: user reference, complaint reference, message, read flag and timestamps. The screenshot evidence shows notification records linked to users and complaints.

Relationships: one citizen can create multiple complaints; a complaint may be assigned to one employee; a user can receive multiple notifications; each notification may refer to a complaint.

9. Important REST API Endpoints

Authentication – base path /api/auth: POST /signup (citizen registration); POST /login (citizen/employee login); POST /admin-login (administrator login); GET /me (current authenticated user).

Complaints – base path /api/complaints: POST / (citizen creates complaint with multipart photo); GET /mine (citizen's complaints); GET /assigned (employee's assigned tasks); GET / (admin complaint list, optional ?status= filter); GET /stats (admin dashboard counts); PATCH /:id/assign (admin assigns employee); PATCH /:id/status (admin or assigned employee updates status).

Employees – base path /api/employees: GET / (list employees and open-task counts); POST / (create employee). Both routes require administrator access.

Notifications – base path /api/notifications: GET / (latest notifications and unread count); GET /unread (unread count); PATCH /read (mark notifications as read). These routes require authentication.

All endpoints are prefixed by the backend API base URL. The exact deployed hostname should be taken from the current Render service settings and frontend environment configuration.

10. Validation and Security

Registration requires name, email and a password of at least six characters; duplicate email addresses are rejected. The user schema stores email in lowercase and enforces uniqueness.

Complaint submission requires a description, an image and location coordinates. Only image MIME types are accepted and the upload limit is 5 MB. The description is limited to 1,000 characters by the schema.

Passwords are hashed using bcryptjs with a cost factor of 10. The password field is excluded from normal Mongoose query results. Login compares the supplied password with the stored hash.

JWTs are signed using the server-side JWT_SECRET and expire after seven days. Role-based middleware restricts citizen, employee and administrator operations.

Deployment security: keep MONGO_URI, JWT_SECRET, ADMIN_EMAIL and ADMIN_PASSWORD in backend environment variables only. Do not expose them through Vite frontend variables or commit .env to GitHub. The public backend repository listing has shown a .env file; remove it from repository tracking and rotate any credentials that were exposed. Add .env to .gitignore and provide only a placeholder .env.example.

11. Testing and Demonstration

Testing should cover normal flows and invalid inputs. The supplied screenshots provide visual evidence of login, registration, complaint submission, citizen report tracking, administrator dashboard, employee assignment and MongoDB records.

Test cases:

1. TC01 – Citizen registration with valid details: expected result is successful account creation and authentication token.
2. TC02 – Registration with missing required fields or password shorter than six characters: expected result is validation error.
3. TC03 – Duplicate email registration: expected result is HTTP 409 and a duplicate-account message.
4. TC04 – Login with incorrect credentials: expected result is HTTP 401 with an invalid-credentials message.
5. TC05 – Complaint without description, photo or coordinates: expected result is HTTP 400 with a validation message.
6. TC06 – Upload a non-image file or image larger than 5 MB: expected result is upload rejection.
7. TC07 – Citizen requests own complaints: expected result is only that citizen’s complaint records.
8. TC08 – Administrator views dashboard: expected result is aggregate counts and complaint list/filtering.
9. TC09 – Administrator assigns a valid employee: expected result is complaint status assigned and notification records for the citizen and employee.
10. TC10 – Employee updates a complaint not assigned to them: expected result is HTTP 403.
11. TC11 – Assigned employee/admin updates status to in_progress or resolved: expected result is updated complaint and a citizen notification for relevant status changes.
12. TC12 – Unauthenticated user accesses a protected endpoint: expected result is authorization failure.

12. Deployment and Installation

Backend repository: https://github.com/Rajit0311/waste-management-server

Frontend repository: https://github.com/Rajit0311/waste-management-frontend

Backend local setup: clone the backend repository; run npm install; create a local .env file with MONGO_URI, JWT_SECRET, ADMIN_EMAIL, ADMIN_PASSWORD and PORT; run npm run seed once to create the initial administrator; run npm run dev for development or npm start for production.

Frontend local setup: clone the frontend repository; run npm install; configure the frontend API base URL using the project's expected Vite environment variable; run npm run dev. For a production build, run npm run build.

Render backend configuration: Root Directory blank when server.js and package.json are in repository root; Build Command npm install; Start Command npm start. Configure secrets in Render Environment Variables. Do not run the seed script as the Build Command because builds may repeat; run the seed command once as a controlled one-off operation or from a trusted local environment.

MongoDB Atlas must allow network access from the deployed backend. Keep the database username/password private and use a least-privilege database user.

13. Results and Output

The supplied output screenshots demonstrate the implemented user-facing workflow: the CleanCity login screen; citizen account creation; report submission with category, description, photo and location; My Reports with a waiting/pending complaint; the administrator dashboard with status summary cards; employee creation and complaint assignment; and MongoDB Compass records for users and notifications.

The screenshots are included in the following section as project output evidence. Sensitive contact details, coordinates and database identifiers have been blurred in this report copy.

14. Conclusion

CleanCity demonstrates how a MERN-stack web application can support local waste complaint reporting and cleanup coordination. Citizens can submit image- and location-supported reports and monitor their status. Administrators can review reports, assign employees and monitor progress. Employees can update the work status, while notifications keep users informed. The project applies React-based UI development, Express REST APIs, MongoDB/Mongoose data storage, authentication, file uploads and role-based access control.

15. Future Scope

Possible future enhancements include live vehicle tracking, route optimisation for collection teams, map-based complaint clustering, automated duplicate-report detection, analytics for ward-wise cleanup performance, SMS/email/push notifications, multilingual support, accessibility improvements, and integration with municipal service systems. These are proposed enhancements and are not claimed as currently implemented.

16. References

1. MERN Stack Project Guidelines supplied for the B.Sc. I.T. Semester V practical project.

2. Backend source repository: https://github.com/Rajit0311/waste-management-server

3. Frontend source repository: https://github.com/Rajit0311/waste-management-frontend

4. React documentation: https://react.dev/

5. Express documentation: https://expressjs.com/

6. MongoDB and Mongoose documentation: https://www.mongodb.com/docs/ and https://mongoosejs.com/docs/

7. Node.js documentation: https://nodejs.org/docs/

Appendix A – Submission Checklist

- Complete frontend and backend source code uploaded to GitHub.
- README.md includes project title, description, technologies and setup instructions.
- .env is excluded from Git tracking; .env.example contains placeholders only.
- MongoDB credentials and JWT secret are stored securely in environment variables.
- CRUD and validation-related flows are demonstrated and tested.
- Both repository links are accessible to the faculty.

Screenshots / Project Output Evidence

The following screenshots were supplied as evidence of the running application and database. Personal contact details, location coordinates and database identifiers have been blurred in this report copy.

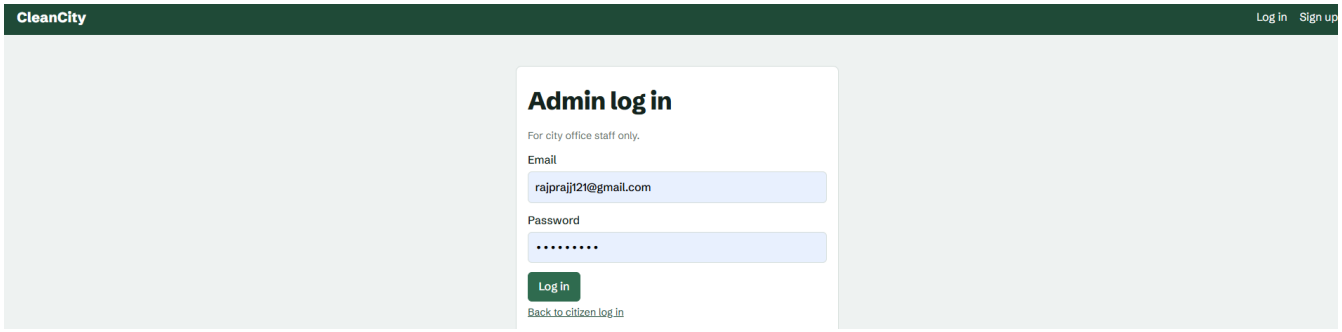


Figure 1. Citizen / employee login screen.

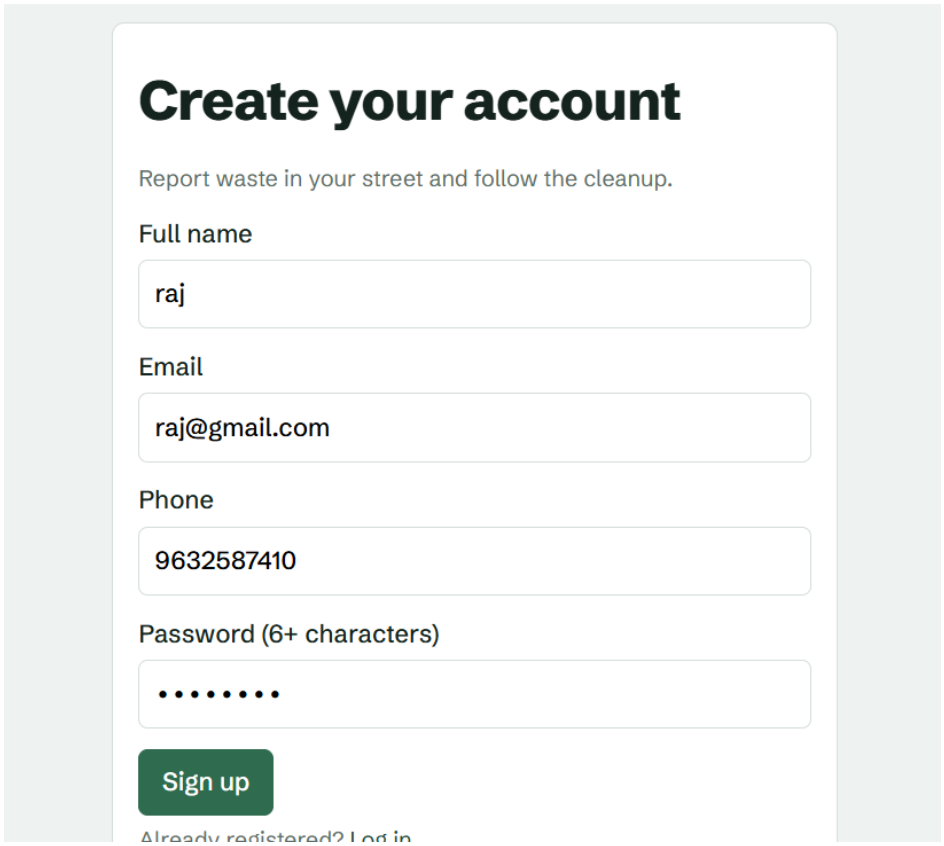


Figure 2. Citizen account registration screen.

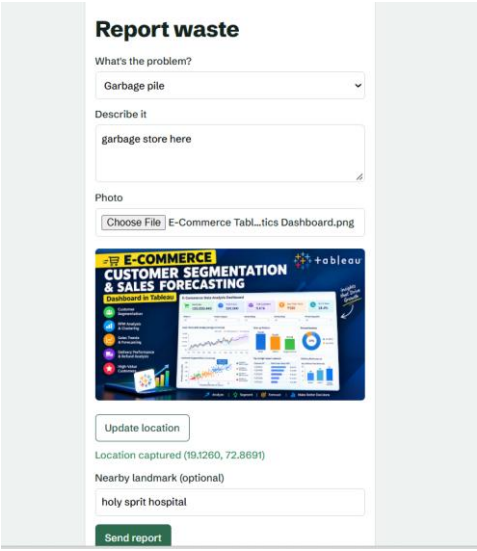


Figure 3. Waste complaint submission with photo and captured location.

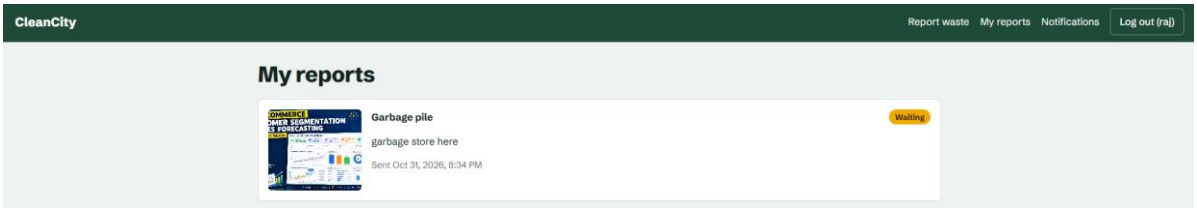


Figure 4. Citizen's My Reports page showing a submitted complaint.

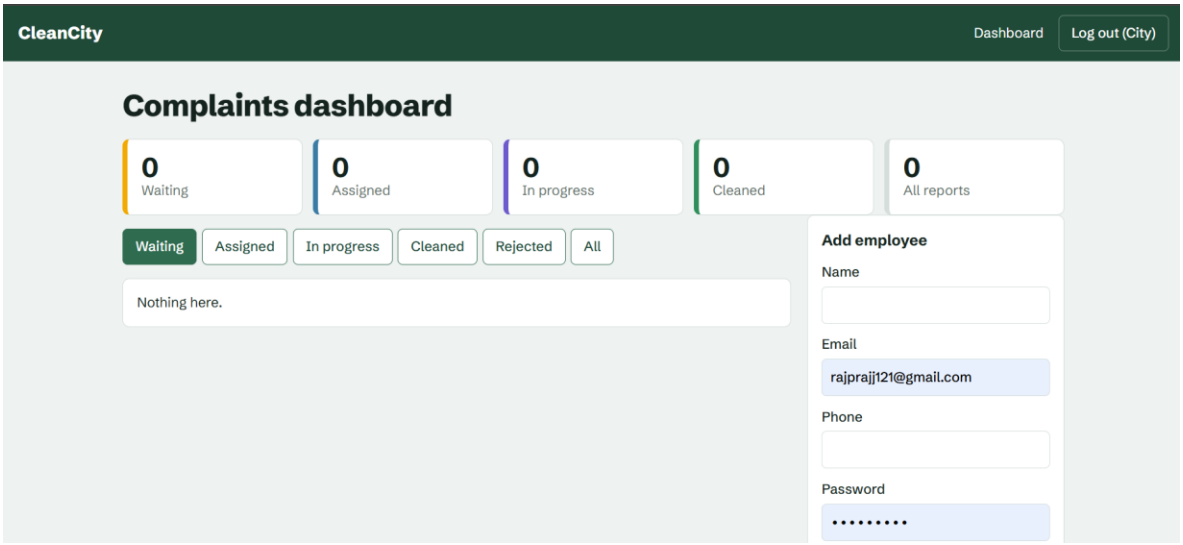


Figure 5. City administrator dashboard before complaints are received.

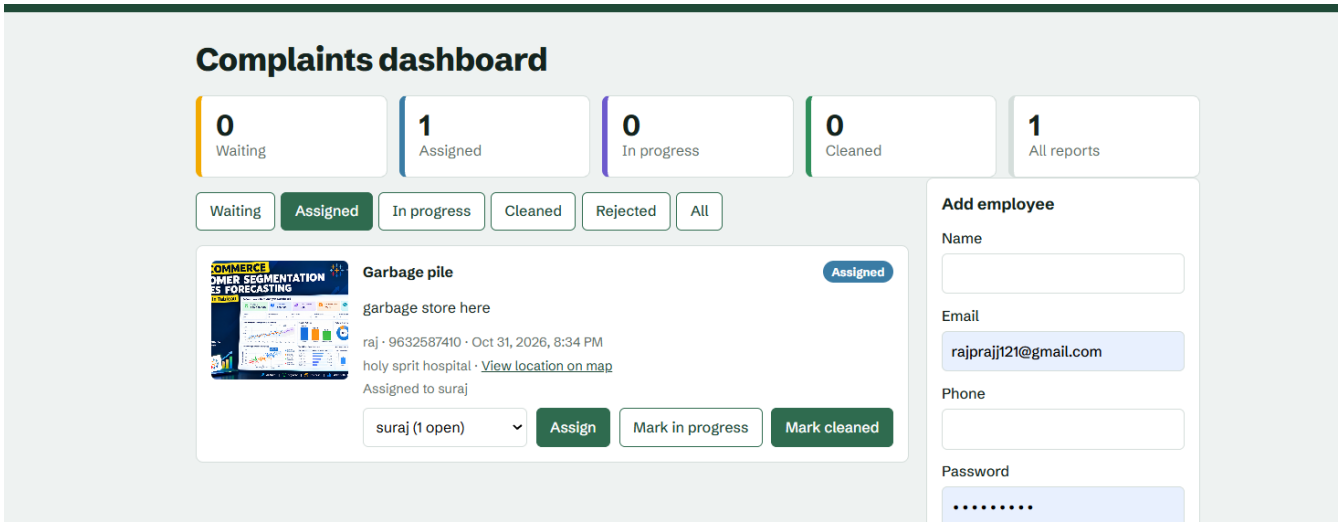


Figure 6. Administrator dashboard showing an assigned complaint and employee assignment controls.

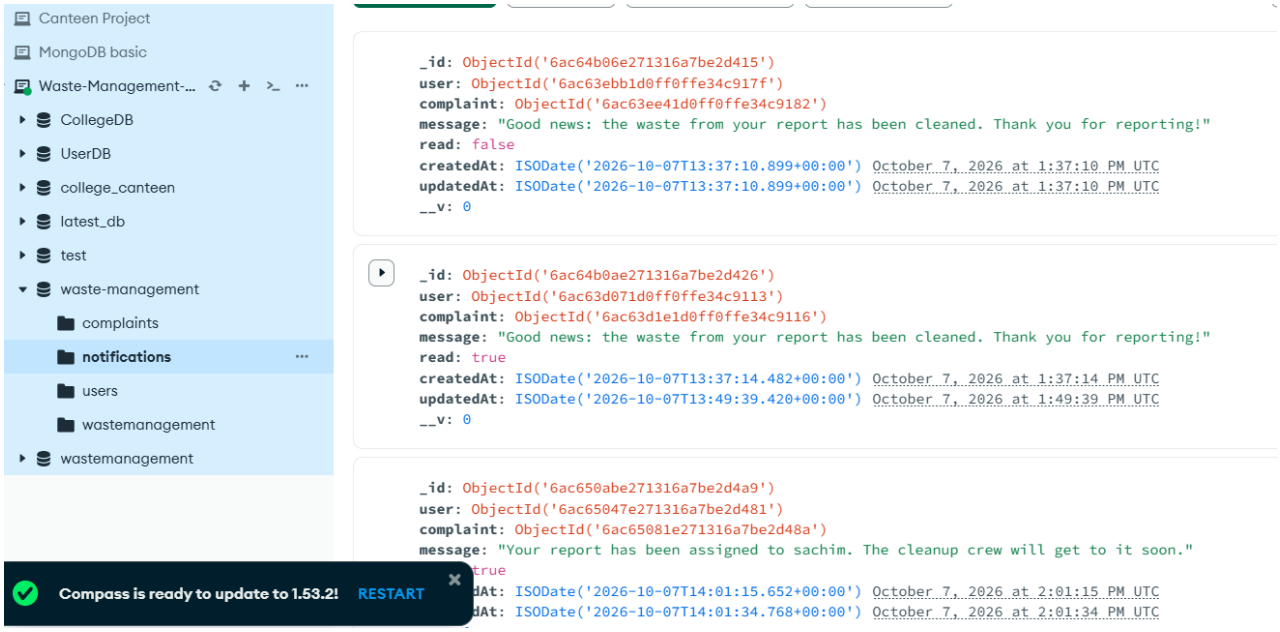


Figure 7. MongoDB Compass users collection evidence (sensitive values blurred).

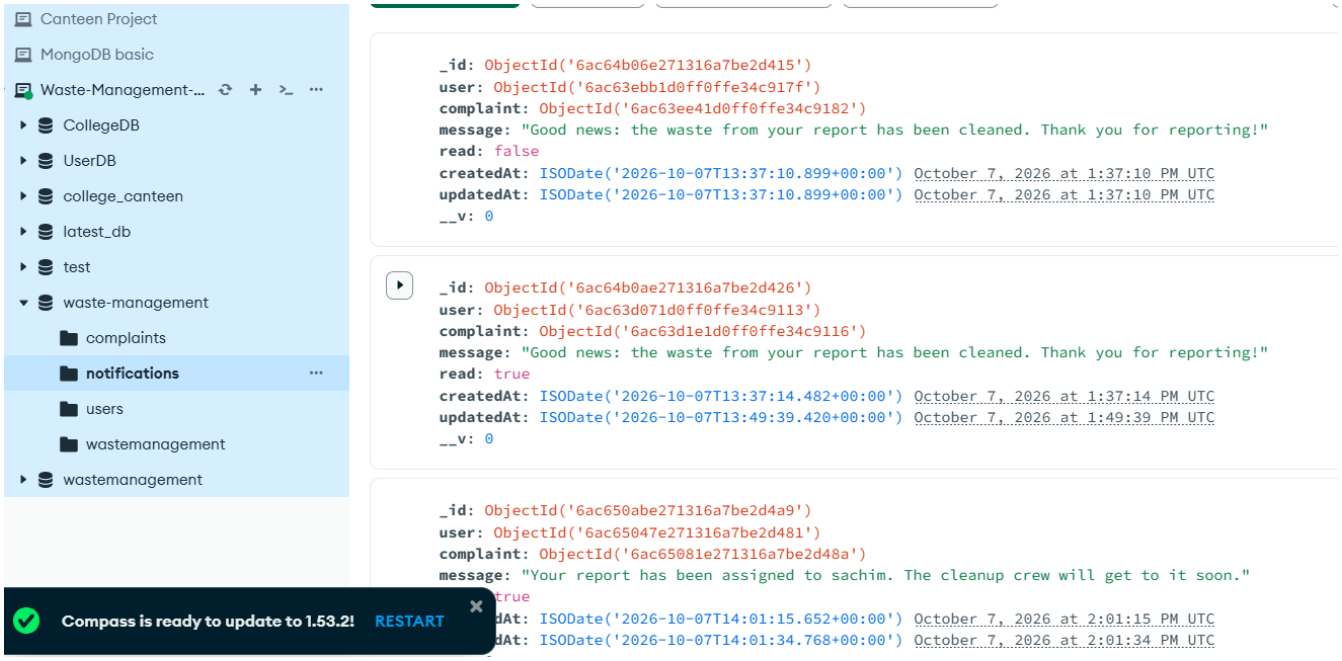


Figure 8. MongoDB Compass notifications collection evidence (sensitive identifiers blurred).